

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего
образования



НИЖЕГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ

УНИВЕРСИТЕТ им. Р.Е.АЛЕКСЕЕВА

Институт радиоэлектроники и информационных технологий

Кафедра «Вычислительные системы и технологии»

ОТЧЕТ

по лабораторной работе №3

по дисциплине

Языки интернет-программирования

РУКОВОДИТЕЛЬ:

(подпись)

Яшнов М.Д.
(фамилия, и.,о.)

СТУДЕНТ:

(подпись)

Ивашина Е.Р.
(фамилия, и.,о.)

23-ИИ
(шифр группы)

Работа защищена «__» _____

С оценкой _____

Ссылка на GitHub: <https://github.com/luft15/Books-club>

Требования к проекту

- Должно быть явное разделение на фронтенд и бэкенд, у каждого свой контейнер
- Использование SPA фреймворка на клиентской стороне
- Развертывание БД через Docker Compose
- Все запросы в БД происходят строго через ORM
- Для генерации схемы БД используются миграции
- Аутентификация через JWT или сессии
- Пароли хэшируются через bcrypt или аналогичные функции с солью
- Контент страницы зависит от прав пользователя. Можно предоставлять данные которые ему принадлежат или давать доступ к разным страницам в зависимости от прав. Самое банальное - админская панель
- Должно быть минимум 2 эндпоинта(помимо установки токена и регистрации), функционал которых требует авторизации. Если будете делать админскую панель, можете на нее впихнуть статистику и графики(запросы на данные к ним через авторизацию)
- Взаимодействие бэкенда и фронтенда через api, api соответствует правилам REST

Ход работы

Первым делом была создана структура проекта, разделена на backend и frontend

Позже был выбран React+Vite:

```

• (venv) PS D:\NGTU\3_c\liz\lp\Books-club> cd .\frontend\
• (venv) PS D:\NGTU\3_c\liz\lp\Books-club\frontend> ls

Каталог: D:\NGTU\3_c\liz\lp\Books-club\frontend

Mode                LastWriteTime         Length Name
----                -
d-----          16.06.2026   13:11             public
-a-----          16.06.2026   13:11             0 Dockerfile

• (venv) PS D:\NGTU\3_c\liz\lp\Books-club\frontend> npm create vite@latest . -- --template react

> npx
> create-vite . --template react

|
| ◇ Current directory is not empty. Please choose how to proceed:
|   Remove existing files and continue
|
| ◇ Install with npm and start now?
|   Yes
|
| ◇ Scaffolding project in D:\NGTU\3_c\liz\lp\Books-club\frontend...
|
| ◇ Installing dependencies with npm...
|
added 135 packages, and audited 136 packages in 17s

31 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
|
| ◇ Starting dev server...
|
> frontend@0.0.0 dev
> vite

VITE v8.0.16 ready in 1214 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
• (venv) PS D:\NGTU\3_c\liz\lp\Books-club\frontend> npm install

added 2 packages, and audited 138 packages in 1s

31 packages are looking for funding

```

```
(venv) PS D:\NGTU\3_c\liz\lp\Books-club\backend\app> mkdir seviles

Каталог: D:\NGTU\3_c\liz\lp\Books-club\backend\app

Mode                LastWriteTime         Length Name
----                -
d-----         16.06.2026   13:15             seviles

(venv) PS D:\NGTU\3_c\liz\lp\Books-club\backend\app> ls

Каталог: D:\NGTU\3_c\liz\lp\Books-club\backend\app

Mode                LastWriteTime         Length Name
----                -
d-----         16.06.2026   13:14             api
d-----         16.06.2026   13:15             core
d-----         16.06.2026   13:15             models
d-----         16.06.2026   13:15             schemas
d-----         16.06.2026   13:15             seviles

(venv) PS D:\NGTU\3_c\liz\lp\Books-club\backend\app> cd ..

Каталог: D:\NGTU\3_c\liz\lp\Books-club\backend

Mode                LastWriteTime         Length Name
----                -
d-----         16.06.2026   13:14             alembic
d-----         16.06.2026   13:15             app
(venv) PS D:\NGTU\3_c\liz\lp\Books-club\backend\app> ls

Каталог: D:\NGTU\3_c\liz\lp\Books-club\backend\app

Mode                LastWriteTime         Length Name
----                -
d-----         16.06.2026   13:14             api
d-----         16.06.2026   13:15             core
d-----         16.06.2026   13:15             models
d-----         16.06.2026   13:15             schemas
d-----         16.06.2026   13:15             seviles
```

На данный момент структура проекта выглядит вот так:

```
(venv) PS D:\NGTU\3_c\liz\lp\Books-club\frontend> cd ..  
(venv) PS D:\NGTU\3_c\liz\lp\Books-club> ls
```

Каталог: D:\NGTU\3_c\liz\lp\Books-club

Mode	LastWriteTime		Length	Name
----	-----		-----	----
d-----	16.05.2026	12:18		.idea
d-----	16.06.2026	13:18		backend
d-----	16.06.2026	13:47		doc
d-----	16.06.2026	13:18		frontend
d-----	30.05.2026	13:43		src
d-----	16.06.2026	13:14		venv
-a----	13.05.2026	15:59	73	.env
-a----	16.05.2026	12:28	96	.gitignore
-a----	16.06.2026	13:21	1252	docker-compose.yml
-a----	13.05.2026	15:41	172	Dockerfile
-a----	16.05.2026	12:11	11	README.md
-a----	16.06.2026	13:14	1786	requirements.txt

С предыдущего задания нужно перенести БД в schema, чтобы автоматически создавать базу данных через alembic.

```

backend > app > core > database.py > ...
1  # backend/app/core/database.py
2  ✓ from sqlalchemy import create_engine
3  from sqlalchemy.ext.declarative import declarative_base
4  from sqlalchemy.orm import sessionmaker
5  import os
6  from dotenv import load_dotenv
7
8  load_dotenv()
9
10 ✓ SQLAlchemy_DATABASE_URL = os.getenv(
11     "DATABASE_URL",
12     "postgresql://postgres:postgres@localhost:5432/bookclub"
13 )
14
15 ✓ engine = create_engine(
16     SQLAlchemy_DATABASE_URL,
17     pool_pre_ping=True,
18     pool_size=10,
19     max_overflow=20
20 )
21
22 SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
23
24 Base = declarative_base()
25
26 ✓ def get_db():
27     db = SessionLocal()
28     ✓ try:
29         yield db
30     ✓ finally:
31         db.close()
32

```

Подключение к базе данных

__init__.py для schema:

```

backend > app > schemas > _init_.py > ...
1  # backend/app/schemas/_init_.py
2  from .user_schema import UserBase, UserCreate, UserLogin, UserResponse, UserInDB
3  from .book_schema import BookBase, BookCreate, BookResponse
4  from .event_schema import EventBase, EventCreate, EventResponse
5  from .booking_schema import BookingBase, BookingCreate, BookingResponse
6  from .token_schema import Token, TokenData, AdminStats
7
8  _all_ = [
9      # User
10     "UserBase", "UserCreate", "UserLogin", "UserResponse", "UserInDB",
11     # Book
12     "BookBase", "BookCreate", "BookResponse",
13     # Event
14     "EventBase", "EventCreate", "EventResponse",
15     # Booking
16     "BookingBase", "BookingCreate", "BookingResponse",
17     # Token
18     "Token", "TokenData", "AdminStats"
19 ]
20

```

Book_schema:

```

backend > app > schemas > book_schema.py > ...
1  # backend/app/schemas/book_schema.py
2  from pydantic import BaseModel, Field, validator
3  from datetime import datetime
4  from typing import Optional
5
6
7  class BookBase(BaseModel):
8      title: str = Field(..., min_length=1, max_length=500, description="Название книги")
9      author: Optional[str] = Field(None, max_length=200, description="Автор")
10     year: Optional[int] = Field(None, ge=0, le=datetime.now().year, description="Год издания")
11
12
13  class BookCreate(BookBase):
14      pass
15
16
17  class BookResponse(BookBase):
18      id: int
19      created_at: datetime
20
21      class Config:
22          from_attributes = True
23

```

Booking_shema:

```

backend > app > schemas > booking_schema.py > ...
1  # backend/app/schemas/booking_schema.py
2  from pydantic import BaseModel, Field
3  from datetime import time, datetime
4  from typing import Optional
5
6  from .event_schema import EventResponse
7
8
9  class BookingBase(BaseModel):
10     event_id: int = Field(..., description="ID события")
11     time_slot: Optional[time] = Field(None, description="Временной слот")
12     comment: Optional[str] = Field(None, max_length=500, description="Комментарий")
13
14
15  class BookingCreate(BookingBase):
16     pass
17
18
19  class BookingResponse(BookingBase):
20     id: int
21     created_at: datetime
22     user_id: int
23     user_name: Optional[str] = None
24     user_email: Optional[str] = None
25     event: Optional[EventResponse] = None
26
27  class Config:

```

event_schema:

```

backend > app > schemas > event_schema.py > ...
1  # backend/app/schemas/event_schema.py
2  from pydantic import BaseModel, Field, validator
3  from datetime import date, datetime
4  from typing import Optional
5
6  from .book_schema import BookResponse
7
8
9  class EventBase(BaseModel):
10     book_id: int = Field(..., description="ID книги")
11     event_date: date = Field(..., description="Дата события")
12     description: Optional[str] = Field(None, max_length=1000, description="Описание")
13     max_participants: int = Field(20, ge=1, le=1000, description="Максимум участников")
14
15
16  class EventCreate(EventBase):
17     @validator('event_date')
18     def validate_date(cls, v):
19         if v < date.today():
20             raise ValueError('Дата события не может быть в прошлом')
21         return v
22
23
24  class EventResponse(EventBase):
25     id: int
26     created_at: datetime
27     book: Optional[BookResponse] = None
28     participants_count: Optional[int] = 0
29
30     class Config:
31         from_attributes = True
32

```

token_schema:

backend > app > schemas > token_schema.py > ...

```
1  # backend/app/schemas/token_schema.py
2  from pydantic import BaseModel
3  from typing import Optional, List, Dict, Any
4
5  from .user_schema import UserResponse
6
7
8  class Token(BaseModel):
9      access_token: str
10     token_type: str = "bearer"
11     user: UserResponse
12
13
14  class TokenData(BaseModel):
15     username: Optional[str] = None
16
17
18  class AdminStats(BaseModel):
19     total_users: int
20     total_books: int
21     total_events: int
22     total_bookings: int
23     popular_events: List[Dict[str, Any]]
24     recent_bookings: List[Dict[str, Any]]
25
```

Chat (CTRL + I) / Share (CTR

user_schema:

```

backend > app > schemas > user_schema.py > ...
1  # backend/app/schemas/user_schema.py
2  from pydantic import BaseModel, EmailStr, Field, validator
3  from datetime import datetime
4  from typing import Optional
5
6
7  class UserBase(BaseModel):
8      username: str = Field(..., min_length=3, max_length=100, description="Имя пользователя")
9      email: EmailStr = Field(..., description="Email адрес")
10     full_name: Optional[str] = Field(None, max_length=200, description="Полное имя")
11     phone: Optional[str] = Field(None, max_length=20, description="Номер телефона")
12
13     @validator('username')
14     def validate_username(cls, v):
15         if not v.replace('_', '').isalnum():
16             raise ValueError('Username может содержать только буквы, цифры и _')
17         return v
18
19
20     class UserCreate(UserBase):
21         password: str = Field(..., min_length=6, max_length=100, description="Пароль")
22
23
24     class UserLogin(BaseModel):
25         username: str = Field(..., description="Имя пользователя")
26         password: str = Field(..., description="Пароль")
27
28
29     class UserResponse(UserBase):
30         id: int
31         is_active: bool
32         is_admin: bool
33         created_at: datetime
34
35         class Config:
36             from_attributes = True
37
38
39     class UserInDB(UserResponse):
40         hashed_password: str
41
42         class Config:
43             from_attributes = True
44

```

Теперь нужно позаботиться об наполнении проекта файлами и перенести старую логику в новую архитектуру, а также мигрировать с Flask на FastAPI

Файл

#backend/seed.py

нужен, чтобы заполнить базу данных некими начальными данными — и целиком и полностью был сгенерирован, потому что это — синтетические данные.

```

backend > seed.py > seed_database
1  # backend/seed.py
2  from sqlalchemy.orm import Session
3  from app.models.user_model import User_model
4  from app.models.book_model import Book_model
5  from app.models.event_model import Event_model
6  from app.models.booking_model import Booking_model
7  from app.services.auth_service import get_password_hash
8  from datetime import date, time, timedelta
9
10
11 def seed_database(db: Session):
12     """Заполнение базы тестовыми данными"""
13
14     # Проверяем, есть ли уже данные
15     if db.query(User_model).first():
16         print("База уже содержит данные, пропускаем сидирование")
17         return
18
19     print("📄 Заполнение базы данных тестовыми данными...")
20
21     # ===== СОЗДАНИЕ ПОЛЬЗОВАТЕЛЕЙ =====
22     print("👤 Создание пользователей...")
23
24     admin = User_model(
25         username="admin",
26         email="admin@bookclub.com",
27         hashed_password=get_password_hash("admin123"),
28         full_name="Администратор Книжного Клуба",
29         is_admin=True,
30         is_active=True
31     )
32     db.add(admin)
33
34     users_data = [
35         ("anna", "Анна Петрова", "+79161234567"),
36         ("ivan", "Иван Сидоров", "+79261234567"),
37         ("maria", "Мария Иванова", "+79221234567"),
38         ("pavel", "Павел Смирнов", "+79171234567"),
39         ("olga", "Ольга Козлова", "+79181234567"),
40     ]
41
42     users = []
43     for username, full_name, phone in users_data:
44         user = User_model(
45             username=username,
46             email=f"{username}@example.com",
47             hashed_password=get_password_hash("password123"),
48             full_name=full_name,

```

Папка models:

Book_model:

```

backend > app > models > book_model.py > ...
1  # backend/app/book.py
2  from sqlalchemy import Column, Integer, String, Text, Date, Time, DateTime, ForeignKey, func
3  from sqlalchemy.orm import relationship
4  from app.core.database import Base
5
6
7  class Book_model(Base):
8      __tablename__ = "books"
9
10     id = Column(Integer, primary_key=True, index=True)
11     title = Column(Text, nullable=False)
12     author = Column(String(200), nullable=True)
13     year = Column(Integer, nullable=True)
14     created_at = Column(DateTime, server_default=func.now())
15
16     # Связь с событиями
17     events = relationship("Event", back_populates="book", cascade="all, delete-orphan")
18

```

booking_model:

```

backend > app > models > booking_model.py > ...
1  # backend/app/booking_model.py
2  from sqlalchemy import Column, Integer, String, Text, Date, Time, DateTime, ForeignKey, func
3  from sqlalchemy.orm import relationship
4  from app.core.database import Base
5
6
7  class Booking_model(Base):
8      __tablename__ = "bookings"
9
10     id = Column(Integer, primary_key=True, index=True)
11     event_id = Column(Integer, ForeignKey("events.id", ondelete="CASCADE"), nullable=False)
12     user_id = Column(Integer, ForeignKey("users.id", ondelete="CASCADE"), nullable=False)
13     time_slot = Column(Time, nullable=True)
14     comment = Column(Text, nullable=True)
15     created_at = Column(DateTime, server_default=func.now())
16
17     # СВЯЗИ
18     event = relationship("Event", back_populates="bookings")
19     user = relationship("User", back_populates="bookings")

```

event_model:

```

backend > app > models > event_model.py > ...
1  # backend/app/event_model.py
2  from sqlalchemy import Column, Integer, String, Text, Date, Time, DateTime, ForeignKey, func
3  from sqlalchemy.orm import relationship
4  from app.core.database import Base
5
6
7  class Event_model(Base):
8      __tablename__ = "events"
9
10     id = Column(Integer, primary_key=True, index=True)
11     book_id = Column(Integer, ForeignKey("books.id", ondelete="CASCADE"), nullable=False)
12     event_date = Column(Date, nullable=False)
13     description = Column(Text, nullable=True)
14     max_participants = Column(Integer, default=20)
15     created_at = Column(DateTime, server_default=func.now())
16
17     # Связи
18     book = relationship("Book", back_populates="events")
19     bookings = relationship("Booking", back_populates="event", cascade="all, delete-orphan")
20

```

user_model:

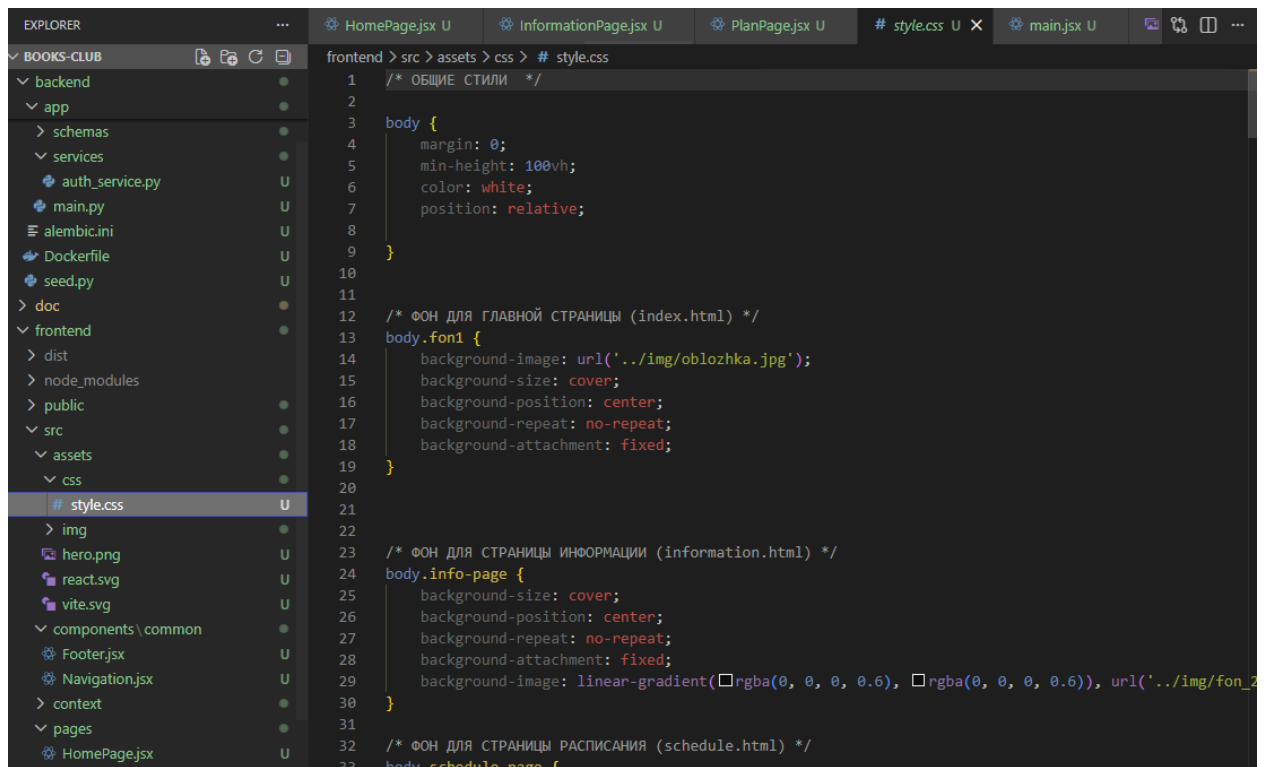
```

backend > app > models > user_model.py > ...
1  # backend/app/user_model.py Chat (CTRL + I) / Share (CTRL + L)
2  from sqlalchemy import Column, Integer, String, Text, Date, Time, DateTime, ForeignKey, Boolean, func
3  from sqlalchemy.orm import relationship
4  from app.core.database import Base
5
6
7  class User_model(Base):
8      __tablename__ = "users"
9
10     id = Column(Integer, primary_key=True, index=True)
11     username = Column(String(100), unique=True, nullable=False, index=True)
12     email = Column(String(200), unique=True, nullable=False, index=True)
13     hashed_password = Column(String(255), nullable=False)
14     full_name = Column(String(200), nullable=True)
15     phone = Column(String(20), nullable=True)
16     is_active = Column(Boolean, default=True)
17     is_admin = Column(Boolean, default=False)
18     created_at = Column(DateTime, server_default=func.now())
19     updated_at = Column(DateTime, onupdate=func.now())
20
21     # Связи
22     bookings = relationship("Booking", back_populates="user", cascade="all, delete-orphan")
23

```

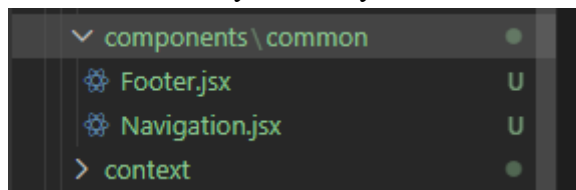
Дальше была проделана работа над frontend

Первом делом перенесены стиль в frontend/src/assets/css/style.css

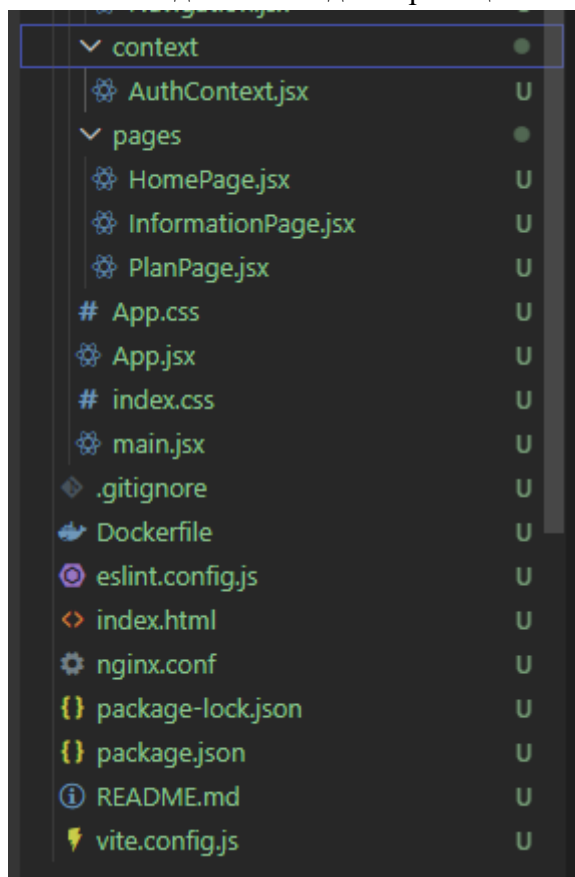


```
1  /* ОБЩИЕ СТИЛИ */
2
3  body {
4      margin: 0;
5      min-height: 100vh;
6      color: white;
7      position: relative;
8  }
9
10
11
12  /* ФОН ДЛЯ ГЛАВНОЙ СТРАНИЦЫ (index.html) */
13  body.fon1 {
14      background-image: url('../img/oblozhka.jpg');
15      background-size: cover;
16      background-position: center;
17      background-repeat: no-repeat;
18      background-attachment: fixed;
19  }
20
21
22
23  /* ФОН ДЛЯ СТРАНИЦЫ ИНФОРМАЦИИ (information.html) */
24  body.info-page {
25      background-size: cover;
26      background-position: center;
27      background-repeat: no-repeat;
28      background-attachment: fixed;
29      background-image: linear-gradient(rgba(0, 0, 0, 0.6), rgba(0, 0, 0, 0.6)), url('../img/fon_2');
30  }
31
32  /* ФОН ДЛЯ СТРАНИЦЫ РАСПИСАНИЯ (schedule.html) */
33  body.schedule-page {
```

Также в отдельную логику были вынесены шапка и футер:



А также отодельно каждая страница:



```

frontend > src > pages > InformationPage.jsx > ...
1 // frontend/src/pages/InformationPage.jsx
2 import React from 'react';
3 import Navigation from '../components/common/Navigation';
4 import Footer from '../components/common/Footer';
5 import './assets/css/style.css';
6 import foto from '../assets/img/foto.jpg';
7 import foto2 from '../assets/img/foto_2.jpg';
8
9 const InformationPage = () => {
10   return (
11     <div className="info-page">
12       <Navigation />
13
14       <main>
15         <div className="container">
16           <article className="article">
17             <div className="article-text">
18               <p>
19                 Книжный клуб «BOOKScub» — это место, где собираются те, кто любит читать и готов
20               </p>
21             </div>
22             <div className="article-with-images">
23               <div className="article-images">
24                 <img src={foto2} alt="Встреча" className="side-image" />
25                 <img src={foto} alt="Книжный клуб" className="side-image" />
26               </div>
27             </div>
28           </article>
29         </div>
30       </main>
31
32       <Footer />
33     </div>
34   );
35 };
36
37 export default InformationPage;
38

```

Стили подключены в frontend/src/main.jsx:

```

frontend > src > main.jsx
1 import { StrictMode } from 'react'
2 import { createRoot } from 'react-dom/client'
3 import './index.css'
4 import App from './App.jsx'
5 import './assets/css/style.css'
6
7 createRoot(document.getElementById('root')).render(
8   <StrictMode>
9     <App />
10   </StrictMode>,
11 )
12

```

Авторизация сделана следующим образом:

JWT (JSON Web Token) — используется для создания и проверки токенов доступа,

OAuth2PasswordBearer — FastAPI-провайдер для получения токена из заголовка,

bcrypt — хэширование паролей (соль + хэш).

Эндпоинты:

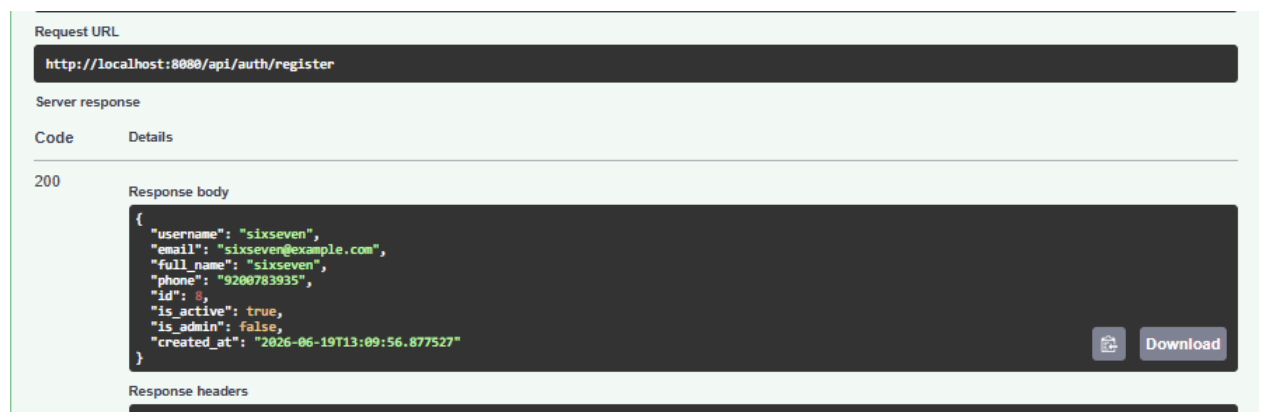
Метод	Путь	Описание
-------	------	----------

POST	/api/auth/register	Регистрация нового пользователя (пароль хэшируется).
------	--------------------	--

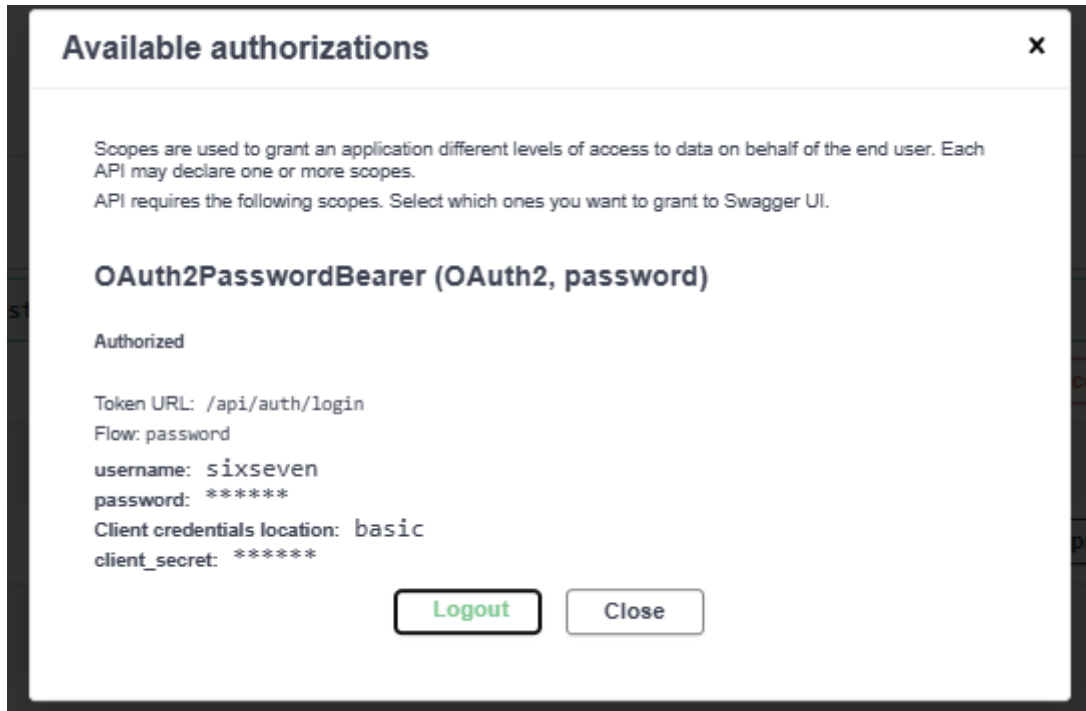
POST	/api/auth/login	Вход (OAuth2-форма) – возвращает access_token.
------	-----------------	--

POST	/api/auth/login-json	Вход (JSON) – возвращает access_token.
------	----------------------	--

GET	/api/auth/me	Получение данных текущего пользователя (требуется токен).
-----	--------------	---



Как видно, ответ 200, значит всё работает. Попробуем авторизоваться



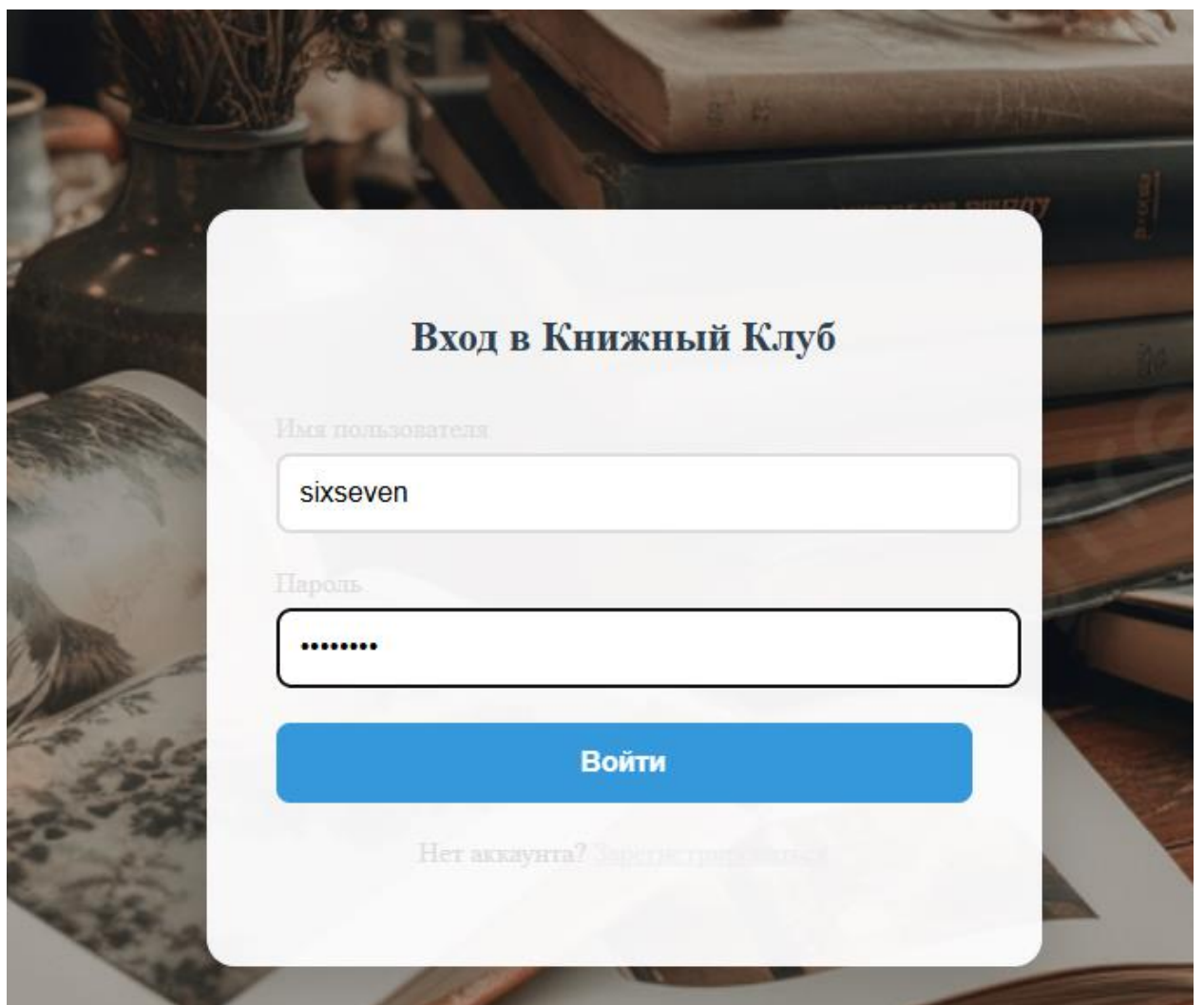
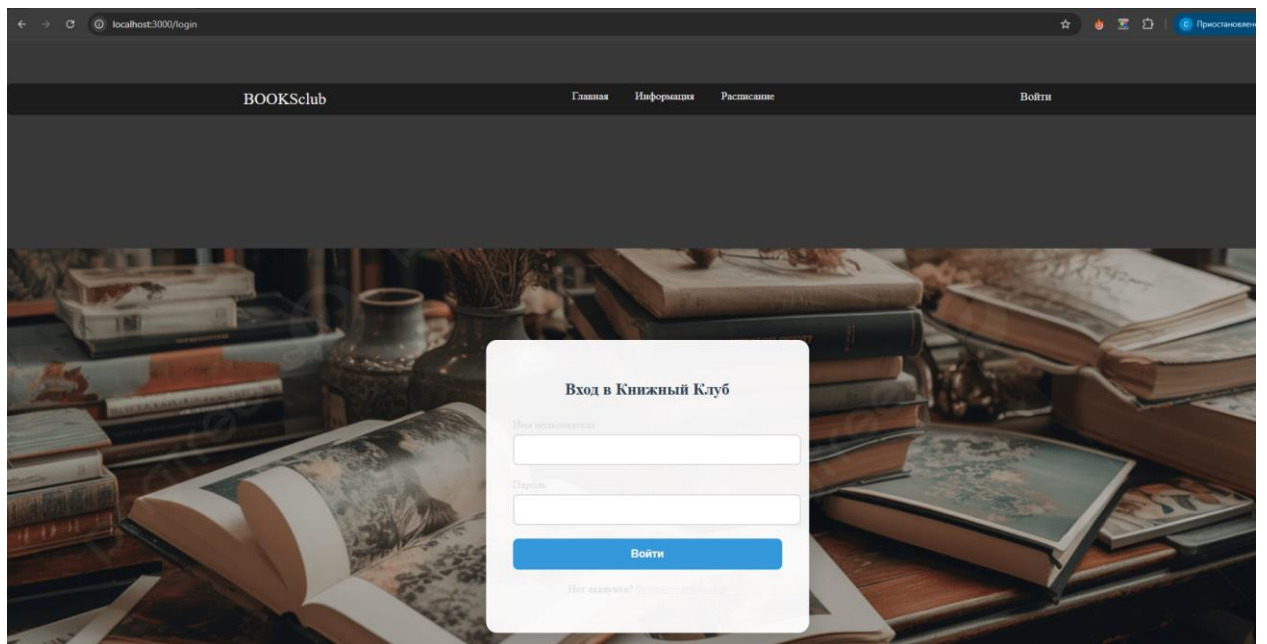
Попробуем эндпоинт /me:

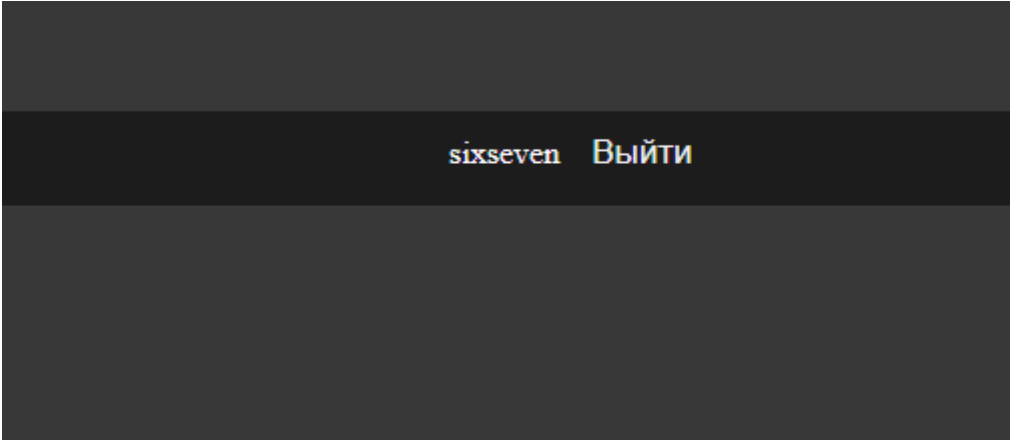


Как видно в теле ответа содержится вся информация о пользователе sixseven

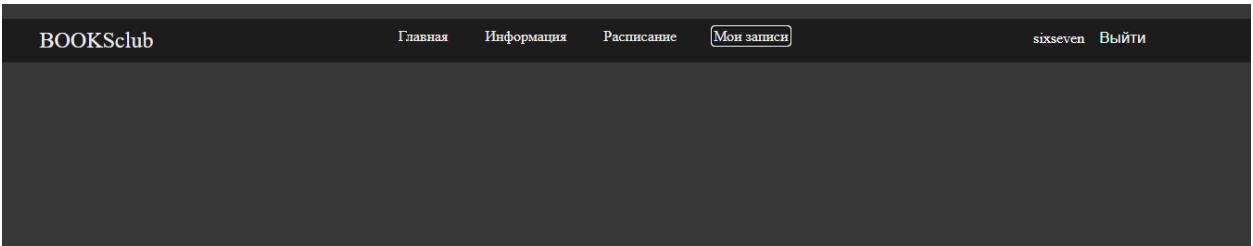
На фронтенде выглядит так:

/login





в хедере справа
появится имя пользователя, а во вкладке Мои записи можно будет увидеть записи на
встречи книжного клуба!



Мои записи

Джейн Эйр

19.07.2026

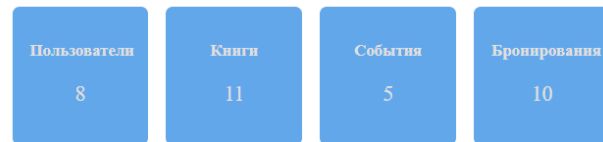
18:00:00

Комментарий: Хочу выступить

Отменить

Админ-панель:

Админ-панель



Создать событие

Книга
Выберите книгу

Дата
ДД.ММ.ГГГГ

Описание

Макс. участников
20

Создать событие

Добавить книгу

Название

Автор

Год

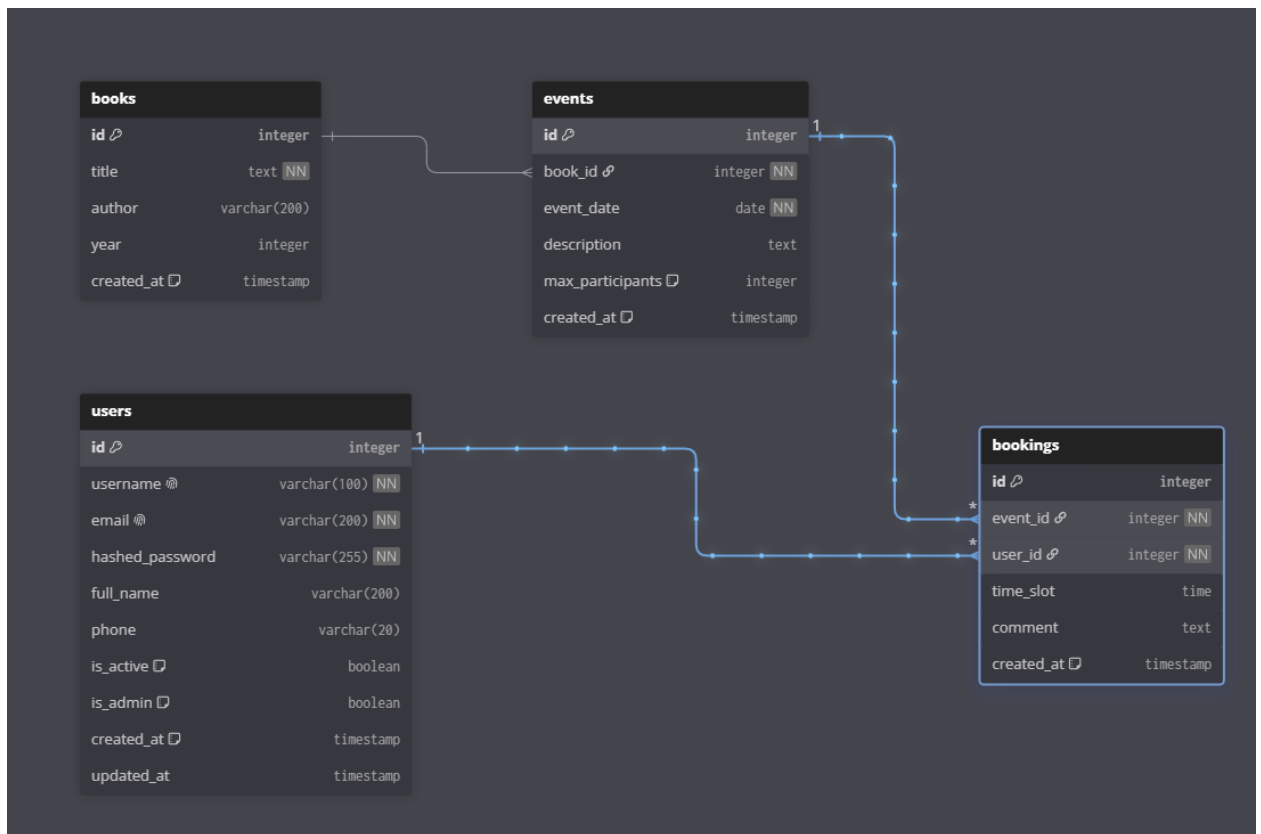
Добавить книгу

Управление событиями

Бесы - 04.07.2026 Офлайн в клубе на ул. Книжная, 10	Удалить
Бесы - 04.07.2026 Офлайн в клубе на ул. Книжная, 10	Удалить

Как видно, тут можно добавлять книги, к которым можно привязывать события, можно создавать встречи(события), а также отменять их, то есть удалять!

ERD-диаграмма:



Для построение ERD-диаграммы был использован сервис dbdiagram.io

Вывод:

В ходе выполнения лабораторной работы был разработан веб-сервис «Книжный клуб» с использованием архитектуры клиент-сервер. Реализован полноценный функционал:

- **Регистрация и аутентификация** пользователей с использованием JWT-токенов, пароли хэшируются с помощью bcrypt.
- **Разделение ролей:**
 - **Обычные пользователи** могут просматривать книги и события, записываться на мероприятия, просматривать свои записи и отменять их.
 - **Администраторы** имеют дополнительный доступ к админ-панели: управление событиями (создание, удаление), добавление новых книг, просмотр статистики (количество пользователей, книг, событий, бронирований).
- **Основные страницы сайта:** главная с ближайшими событиями, страница информации о клубе, страница расписания с календарем и формой записи, профиль пользователя, админ-панель.

- **API-эндпоинты** соответствуют правилам REST, часть из них защищена авторизацией (запись на событие, просмотр своих записей, административные действия).
- Взаимодействие с базой данных осуществляется через ORM (SQLAlchemy), схема БД создается через миграции (Alembic).
- Проект полностью контейнеризирован (Docker Compose) и состоит из трёх сервисов: база данных (PostgreSQL), бэкенд (FastAPI) и фронтенд (React SPA).

Все требования, предъявляемые к работе, выполнены в полном объёме.